

Análisis de Sistemas

Materia:
Análisis y Metodología de
Sistemas

Docente contenidista: QUIRÓZ, Gamaliel

Revisión: Coordinación



Contenido

Introducción	04
Historia de la Inteligencia Artificial	04
Modelos de Inteligencia Artificial.....	06
Librerías y herramientas prácticas de IA	08
¿Qué es scikit-learn?	08
Integrando IA con Python	11
Bibliografía	21

Clase 10



iTe damos la bienvenida a la materia
Análisis y Metodología de Sistemas!

En esta clase vamos a ver los siguientes temas:

- La historia de la IA, desde Turing hasta los LLMs multimodales.
- Los modelos principales: simbólicos, machine learning, redes neuronales y deep learning.
- Herramientas prácticas como scikit-learn, CountVectorizer y Naive Bayes, y cómo integrarlas con Flask.

Introducción

En esta clase nos adentraremos en uno de los campos más transformadores de la informática moderna: la Inteligencia Artificial (IA). Desde sus raíces históricas hasta sus aplicaciones más actuales, la IA ha evolucionado a partir de reglas básicas escritas por humanos hacia sistemas capaces de aprender, adaptarse y generar conocimiento nuevo.

El objetivo es comprender cómo nació la IA, cómo ha progresado por décadas, cuáles son los principales modelos utilizados, y qué herramientas permiten implementar hoy en proyectos reales. Además, iremos vinculando la teoría con ejemplos prácticos, preguntas para reflexionar y actividades que faciliten la comprensión.

Historia de la Inteligencia Artificial

Para entender la IA de hoy, necesitamos viajar al pasado. Su historia refleja una mezcla de ambición científica, avances tecnológicos y limitaciones propias de cada época.

Origen e hitos históricos

1950 – Alan Turing y el Test de Turing

- Pregunta fundacional: ¿Pueden las máquinas pensar?
- El test de Turing proponía una prueba de imitación: si una máquina logra conversar con un humano sin ser distinguida, podríamos considerarla “inteligente”.

1956 – Conferencia de Dartmouth

- John McCarthy acuñó el término Inteligencia Artificial, marcando el inicio oficial de la disciplina.



¿Qué opinas? ¿Una máquina que engaña a un humano en una conversación realmente "piensa"?

IA por décadas

Años 60–70 – Sistemas Expertos.

Se crean los primeros sistemas expertos, como DENDRAL y MYCIN, que podían simular la toma de decisiones de un experto humano en áreas específicas (como diagnóstico médico). Estos sistemas usaban reglas escritas a mano (si ocurre A, hacer B), y aunque eran muy buenos en dominios limitados, no podían aprender ni adaptarse.

DENDRAL (química) y MYCIN (medicina).

- Ventaja: podían emular decisiones humanas en campos específicos.
- Desventaja: no aprendían por sí mismos.

Años 80–90 – Aprendizaje Automático

Aparece el **Machine Learning (aprendizaje automático)**: los sistemas ya no dependen sólo de reglas, sino que **aprenden patrones desde los datos**.

- Surgen algoritmos como Árboles de Decisión, KNN, Naive Bayes, y redes neuronales básicas.
- Cambio clave: los sistemas aprenden de datos, no sólo de reglas.

2010 en adelante – Deep Learning

Con el aumento de poder computacional y la disponibilidad de grandes volúmenes de datos, surgen las redes neuronales profundas (Deep Learning). Modelos como las CNN (Convolutional Neural Networks) y LSTM permiten grandes avances en:

- Avance gracias al poder computacional y grandes volúmenes de datos.
- Aplicaciones: reconocimiento de imágenes, traducción automática, asistentes de voz.

2018–hoy – Modelos de Lenguaje Grande (LLMs)

- GPT, BERT, Claude, LLaMA, Gemini.
- Capacidad de escribir, razonar, traducir, analizar texto y procesar múltiples modalidades (texto, imágenes, voz).

Modelos de Inteligencia Artificial

Modelos tradicionales o simbólicos

- Basados en reglas escritas manualmente.

Ejemplo: un chatbot que responde solo a palabras clave.

- Útiles en tareas simples y bien definidas.

¿Qué limitaciones tendría un chatbot basado solo en reglas en comparación con ChatGPT?

Aprendizaje automático (Machine Learning)

a) Supervisado – Aprende con datos etiquetados.

Ejemplo: clasificador de sentimientos.

b) No supervisado – Encuentra patrones ocultos.

Ejemplo: segmentación de clientes en marketing.

- c) Por refuerzo – Aprende por prueba y error.

Ejemplo: un robot que aprende a caminar.

¿Cómo aplicarías un modelo supervisado en una escuela para predecir el rendimiento de los alumnos?

Redes neuronales y Deep Learning

- Inspiradas en el cerebro humano.
- Potentes imágenes, voz y texto.
- CNN → imágenes (ej: diagnóstico médico).
- RNN / LSTM → secuencias (ej: traducción).
- Transformers → base de los LLMs.

Modelos de lenguaje grande (LLMs)

- Entrenados con billones de palabras.

Ejemplos: ChatGPT, Gemini, Claude, LLaMA.

- Aplicaciones: chatbots, resúmenes, generación de código.

Librerías y herramientas prácticas de IA

¿Qué es scikit-learn?

scikit-learn es una librería de código abierto para **Python** especializada en aprendizaje automático (Machine Learning).

Ofrece una gran variedad de algoritmos y herramientas para la construcción de modelos predictivos y de análisis de datos, permitiendo a los desarrolladores e investigadores aplicar técnicas de clasificación, regresión, clustering y reducción de dimensionalidad de manera sencilla y eficiente. Su diseño busca ser accesible, modular y reutilizable, por lo que es ampliamente utilizada tanto en entornos académicos como en proyectos industriales.

Características principales:

- Incluye algoritmos de aprendizaje supervisado (como regresión logística, máquinas de soporte vectorial, árboles de decisión).
- Implementa métodos de aprendizaje no supervisado (clustering con K-Means, DBSCAN, reducción de dimensionalidad con PCA).
- Dispone de utilidades para procesamiento de datos, como normalización, selección de características y validación cruzada.
- Está construida sobre NumPy, SciPy y matplotlib, lo que la hace compatible con el ecosistema científico de Python.

CountVectorizer

CountVectorizer es una clase de la librería **scikit-learn** que se utiliza para transformar un conjunto de documentos de texto en una matriz numérica de frecuencias de palabras. Esta técnica se conoce como "bolsa de palabras" (Bag of Words), en la que el orden de las palabras no importa, sino únicamente la frecuencia de aparición de cada una en los textos analizados.

Funcionamiento:

1. Construye un vocabulario con todas las palabras únicas que aparecen en el corpus.
2. Representa cada documento como un vector, donde cada posición corresponde a una palabra del vocabulario y su valor indica el número de veces que aparece en ese documento.
3. El resultado es una matriz dispersa, donde las filas representan documentos y las columnas representan palabras.

Ejemplo simplificado:

- Frases: ["me encanta esta clase", "esto es aburrido"]
- Vocabulario: ["me", "encanta", "esta", "clase", "esto", "es", "aburrido"]

Matriz Resultado:

- [1,1,1,1,0,0,0] → "me encanta esta clase"
- [0,0,0,0,1,1,1] → "esto es aburrido"

MultinomialNB (Naive Bayes Multinomial)

MultinomialNB es un clasificador estadístico de la familia Naive Bayes que se basa en el cálculo de probabilidades, diseñado especialmente para manejar datos de tipo discreto (como el conteo de palabras en documentos de texto). Se fundamenta en el Teorema de Bayes, que calcula la probabilidad de que un documento pertenezca a una categoría en función de la frecuencia de aparición de las palabras.

Características:

- Asume que las características (palabras) son independientes entre sí, lo cual rara vez ocurre en el lenguaje natural, pero aun así ofrece resultados muy efectivos.
- Funciona muy bien en clasificación de texto, como detección de spam, análisis de sentimientos o categorización de noticias.
- Su entrenamiento es rápido y requiere pocos recursos, lo que lo hace ideal para prototipos y sistemas en producción.

Ejemplo conceptual:

Si en los textos positivos aparece mucho la palabra “bueno” y en los negativos la palabra “malo”, MultinomialNB utilizará esas frecuencias para clasificar un nuevo texto.

Integración con Flask

Sabemos que Flask es un micro-framework de Python utilizado para desarrollar aplicaciones web. Al integrarlo con modelos de Machine Learning entrenados en **scikit-learn**, es posible construir aplicaciones interactivas que reciban datos de usuarios a través de un navegador y devuelvan predicciones en tiempo real.

Funcionamiento en la práctica:

1. Se entrena un modelo de Machine Learning en Python con scikit-learn.
2. Ese modelo se guarda o se incorpora dentro de una aplicación Flask.
3. Flask expone una interfaz web (endpoints) donde los usuarios pueden enviar texto o datos.

El modelo procesa la entrada y devuelve la predicción al usuario (ejemplo: “positivo” o “negativo”).

Ejemplo:

Un sistema de análisis de reseñas en línea donde el usuario escribe un comentario y la aplicación, mediante Flask y un clasificador Naive Bayes entrenado con CountVectorizer, responde automáticamente si el comentario es positivo o negativo.

Integrando IA con Python



Ahora veremos cómo podemos integrar estas cosas que aprendimos en un proyecto usando scikit-learn, flask y Python.

Primer Paso

Primeramente es importante crear nuestro entorno virtual, lo hacemos cómo hemos visto en clases anteriores, usando la terminal y ejecutando los comandos en el siguiente orden:

1. **py -m venv venv** (Creamos el entorno).
2. **pip install --upgrade pip** (Instalamos dependencias de gestor de paquetes).
3. **pip install flask scikit-learn joblib flask-cors** (Instalamos los paquetes de scikit-learn).

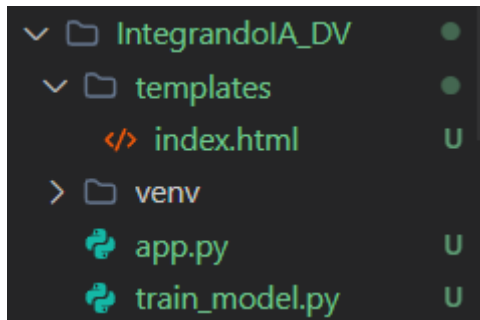
```
py -m venv venv  
pip install --upgrade pip
```

```
pip install flask scikit-learn joblib flask-cors
```

Segundo Paso

Estructura del proyecto

Para buenas prácticas y que todo funcione correctamente, deberíamos tener entonces nuestro proyecto con la siguiente estructura:



Posteriormente se crearán archivos una vez ejecutamos nuestros códigos.

Tercer paso

Entrenar y guardar el modelo (train_model.py)

Link de acceso al archivo train_model.py:

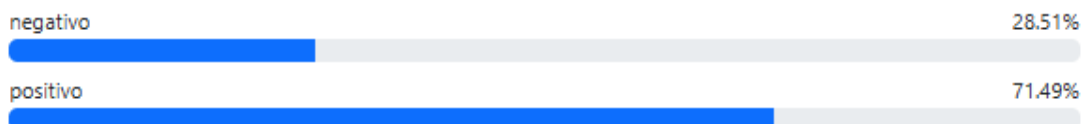
https://docs.google.com/document/d/1EQ9kOVN4ZpN9aTn-BHJVX1dgZ6HkQFKvfNC9C_sv-oo/edit?usp=sharing

Analizador de Sentimiento

Escribí una frase en español y el modelo te dirá si es **positivo** o **negativo**.

Resultado: **positivo**

Probabilidades:



Historial reciente

«Me encanta este curso»	negativo: 28.51%
positivo	positivo: 71.49%
«El día estuvo lindo»	negativo: 42.86%
positivo	positivo: 57.14%
«Me gusta salir»	negativo: 43.61%
positivo	positivo: 56.39%

Explicación de métodos:

- **CountVectorizer():** transforma colecciones de textos en una matriz de conteos por palabra (bag-of-words).
- **MultinomialNB():** clasificador probabilístico adecuado para conteos de palabras.
- **Pipeline([...]):** combina transformaciones y estimador en un solo objeto; facilita fit, predict y serialización.
- **train_test_split(...):** divide datos para evaluar rendimiento básico.

- **pipeline.fit(...):** entrena vectorizador y clasificador en conjunto.
- **dump(pipeline, "sentiment_model.joblib"):** guarda el Pipeline (vectorizador + modelo) en un solo archivo para usar en Flask.

Los texts/labels los podemos reemplazar por un CSV real con muchas muestras, de esta forma podemos comparar CountVectorizer() vs TfidfVectorizer() y variando ngram_range=(1,2).

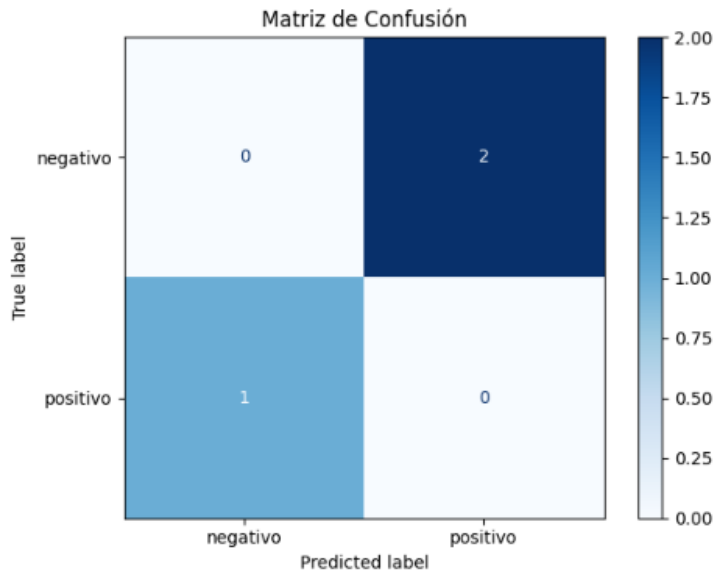
Cuarto paso

Servidor Flask con formulario y API (app.py)

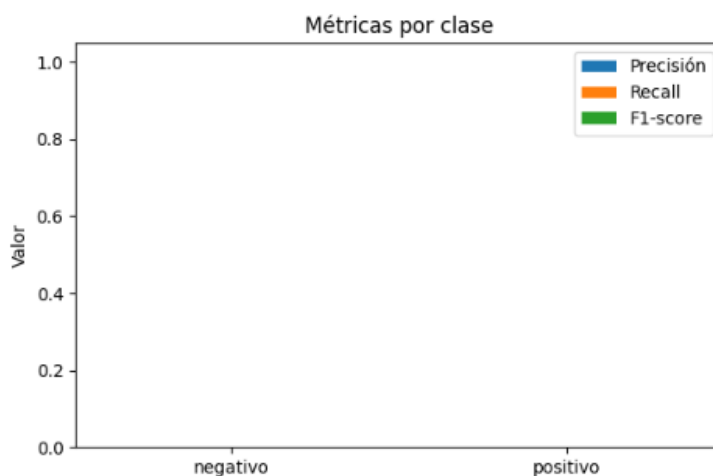
Link de acceso al archivo app.py:

<https://docs.google.com/document/d/1hfgEkcLgUHOEfRnTCckj21Zgwe5huWEAxB1v9LX0kMI/edit?usp=sharing>

Matriz de Confusión



Métricas por clase



Explicación:

- **model = load(MODEL_PATH):** cargamos el pipeline serializado una sola vez cuando arranca la app (muy importante para rendimiento).
- Ruta **"/ (index)":** muestra el formulario HTML.
- Ruta **"/predict":** maneja el formulario (método POST), toma text y devuelve HTML con la predicción y probabilidades.

- **model.predict([text]):** devuelve la etiqueta predicha.
- **model.predict_proba([text]):** devuelve probabilidades por clase (si el clasificador lo soporta — MultinomialNB sí).
- Ruta **"/api/predict"**: ofrece una API JSON (para pruebas con curl, Postman o fetch desde JS).
- **CORS(app):** opcional, facilita peticiones desde otros orígenes en ejercicios frontend.

Quinto paso

Plantilla HTML simple (templates/index.html)

Link de acceso al archivo index.html:

<https://docs.google.com/document/d/1I8A7oFqFmkle8MRsvro0yd9T8ABdPMi0J3q6FW8mqmI/edit?usp=sharing>

Sexto paso

Orden de comando para probar los archivos

Entrenar y guardar:

```
> py train_model.py
```

Veremos lo siguiente una vez lo ejecutamos:

```
Modelo guardado en sentiment_model.joblib
```

Se crea el archivo y veremos "Modelo guardado en sentiment_model.joblib"

Ejecutar servidor Flask (ejecutamos el app.py):

```
py app.py
```

Se levanta nuestro servidor y ya podemos probarlo, deberíamos ver nuestra página de la siguiente manera:

Analizador de Sentimiento

Escribí una frase en español y el modelo te dirá si es **positivo** o **negativo**.

Probar api con postman

En el hards, para la clave y el valor especificamos lo siguiente:

POST ⌵ http://127.0.0.1:5000/api/predict

Params Authorization Headers (10) Body ● Scripts ● Settings

Headers 👁 9 hidden

	Key	Value
☒	Content-Type	application/json
	Key	Value

Probamos enviando con el método **POST** el json con el texto y veremos que nos devuelve:

```
1 {
2   "text": "me encanta esta clase"
3 }
4 |
```

Body Cookies Headers (6) Test Results (1/1) ↻

{ } JSON ▾ ▶ Preview 🔗 Visualize ▾

```
1 {
2   "prediction": "positivo",
3   "probabilities": {
4     "negativo": 0.1239,
5     "positivo": 0.8761
6   },
7   "text": "me encanta esta clase"
8 }
```

Resumen y explicación general de lo que hicimos (qué comprobar y por qué):

- Carga del modelo en inicio: evita read/load por petición - mejora latencia y evita corrupción por concurrencia.
- Pipeline: encapsula vectorizador+modelo. Al guardar/recuperar el pipeline ya viene con el vocabulario aprendido por CountVectorizer.
- predict_proba: útil para entender confianza; no siempre usar el label directo sino mostrar prob.
- Validación: comparar accuracy, precision, recall, f1 en train_model.py con un set más amplio.
- Mejoras fáciles para prácticas:
- Reemplazar CountVectorizer por TfidfVectorizer.
- Probar ngram_range=(1,2) o stop_words="spanish".
- Guardar y mostrar la matriz de confusión.
- Incluir preprocesado: lowercasing, quitar signos de puntuación, lematización.

Códigos explicados (resumen conciso)

- **train_model.py:**

Prepara datos → divide en train/test → crea Pipeline (CountVectorizer + MultinomialNB) → fit → evalúa → dump.

- **app.py:**

Carga joblib.load pipeline al iniciar → define rutas / (form), /predict (form POST), /api/predict (JSON) → usa model.predict y model.predict_proba → devuelve HTML o JSON.

- **index.html:**

Formulario que envía texto a /predict y muestra prediction y probs.

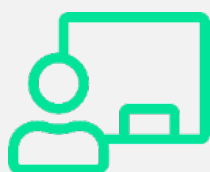
¡Y hasta acá la clase de hoy!

La evolución de la IA refleja el camino de la informática: de reglas simples a sistemas capaces de aprender y razonar. Hoy, la IA no solo es una disciplina académica, sino una tecnología cotidiana con impacto en medicina, educación, industria y comunicación.



Hemos llegado así al final de esta clase en la que vimos:

- La historia de la IA, desde Turing hasta los LLMs multimodales.
- Los modelos principales: simbólicos, machine learning, redes neuronales y deep learning.
- Herramientas prácticas como scikit-learn, CountVectorizer y Naive Bayes, y cómo integrarlas con Flask.



Te esperamos en la **clase en vivo** de esta semana.
No olvides realizar el **desafío semanal**.

¡Hasta la próxima clase!

Bibliografía

- Russell, S., & Norvig, P. (2021). *Artificial intelligence: A modern approach* (4th ed.). Pearson.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
- Mitchell, T. M. (1997). *Machine learning*. McGraw-Hill.
- Chollet, F. (2021). *Deep learning with Python* (2nd ed.). Manning Publications.
- Stanford Encyclopedia of Philosophy. (2020). *Artificial intelligence*. Stanford University. <https://plato.stanford.edu/entries/artificial-intelligence/>
- scikit-learn Developers. (2024). *Scikit-learn: Machine learning in Python*. <https://scikit-learn.org/>